数据库内核实验手册

数据库系统设计与实现 | 2027年度 | Ferrum C++ 实验主线

本手册面向数据库内核实验，按 L0 至 L7 共 32 学时组织。每项实验明确实现范围、输入、预期结果、失败定位和提交证据。代码工作在 Linux 或 WSL2 本地目录进行；报告和正式结果再归档，避免在云盘同步目录中编译和运行数据库。

实验	学时	课堂核心	当前自动验收
L0	2	环境准备与源码阅读	环境与给定原语入口
L1	4	缓冲池与记录管理	独立注册或整库接线待补齐
L2	6	B+树与唯一索引	独立注册或整库接线待补齐
L3	6	Append Unique 与查询执行	Append 6项和Unique 10项
L4	4	事务锁与死锁处理	独立注册或整库接线待补齐
L5	4	快照与多版本可见性	独立注册或整库接线待补齐
L6	4	WAL 重做与恢复验证	独立注册或整库接线待补齐
L7	2	集成验收与性能分析	独立注册或整库接线待补齐

实验按源码阅读、状态推演、实现与测试四步开展。各模块验收安排见上表。

通用提交规则

每次实验提交源代码 diff、本地提交号、环境、完整命令、原始日志和结果摘要。结果状态分别写通过、失败、未执行、阻塞；保留失败日志。报告中的预期结果与实测结果分列，附上可复制的输入。涉及参考或 AI 辅助时说明使用范围，并能够独立解释和修改自己的实现。

课堂 32 学时用于指导与验收，开发练习在课外完成。扩展题、完整 83 题和满规模决赛训练由课外竞赛安排承接。课程成绩按开课时公布的大纲与评分细则执行，比赛成绩单独记录。

阅读目录

L0 环境准备与源码阅读 3

L1 缓冲池与记录管理 4

L2 B+树与唯一索引 6

L3 Append Unique 与查询执行 7

L4 事务锁与死锁处理 8

L5 快照与多版本可见性 10

L6 WAL 重做与恢复验证 11

L7 集成验收与性能分析 13

附录一 实验报告与答辩 15

材料依据与进一步阅读 16

L0 环境准备与源码阅读

建议课内学时 2

目标是建立可重复的开发与验收流程，读懂一条查询经过的模块。本实验从起始框架开始。学生版的大量算法区域故意挖空，明确的“未实现”失败与工具链错误要分开记录。

准备材料与环境

使用 04_Ferrum实验基础 中清理后的 Ferrum-cpp-student.tar.gz，在 Linux 或 WSL2 的本机文件系统解压。源码构建和数据库文件使用本机磁盘目录。工具合同为 C++23、GNU g++ 13 – 15、CMake 3.22 以上、Ninja 和 Python 3.10 以上；dev.sh 提供 Debian 13/GCC 14 的 Docker 环境。先安装并启动 Docker。

```
mkdir -p ~/db-kernel-2027
cd ~/db-kernel-2027
# 将学生源码包复制到本目录后执行
tar -xzf Ferrum-cpp-student.tar.gz
cd Ferrum-cpp-student
mkdir -p course-results/L0
export LABS_JOBS=2
./dev.sh list --stage 06
./dev.sh scaffold --report course-results/L0/scaffold.json
```

运行器默认构建并执行给定原语冒烟，不替学生补算法。Docker 镜像下载、编译器版本不符、缺少头文件和算法待实现是不同错误。首次运行可能较慢；提交首个根因及其上下文，不用最后一行“构建失败”代替诊断。

建立代码身份

```
git init
git add .
git commit -m "Record supplied student baseline"
git rev-parse HEAD
git status --short
./dev.sh exec g++ --version
./dev.sh exec cmake --version
./dev.sh exec python3 --version
```

以上 Git 命令在新解压目录执行。若 Git 尚未配置作者身份，使用自己的课程身份配置后再提交。发放包使用独立的本地 Git 历史；报告记录源码包 SHA256 和本地提交号。

源码阅读步骤

先读 README.md、docs/course/conventions.md 和 labs/README.md。沿 libs/engine 的 Database/Session、libs/planner 与 libs/execution 阅读声明和调用。打开 labs/manifest.json，找出 S06.Q02 的路径、区域和测试名，再定位 LAB 标记。每个学生只负责被分配题号的区域，共享文件不意味着整文件都可以改写。

```
rg -n 'LAB-S06\.Q02:.*-BEGIN' libs
./dev.sh unit S06.Q02 --report course-results/L0/append-initial.json
./dev.sh exec python3 -m unittest discover -s labs/tests -v
```

Append 初始算法桩失败是预期结果。运行器自身测试检查调度与报告逻辑。若单题返回注册或环境错误，按环境、注册和算法三类定位。

课内安排与提交

时间分配	完成内容
0.5 学时	解压、记录环境和版本
0.5 学时	运行 scaffold 并分类首个错误
1 学时	源码路径图、初始失败记录与 C++ 所有权练习

提交 environment.txt、源码包哈希、本地提交号、原始构建日志、scaffold 与 Append 报告，以及一页查询调用路径图。另写一个 unique_ptr 移动与借用视图失效的短例子，解释所有者何时销毁，并标出借用视图的有效期。

验收与答辩

验收检查环境记录能否复现、错误分类是否有依据、调用路径是否能定位到真实文件。答辩解释 pin 与页锁的区别、移动后对象的使用边界，以及 plan 命令生成的题集与依赖。记录表中保留“通过、失败、未执行、阻塞”四种状态。

源码题目：S06.Q02 Append 流式串接

L1 缓冲池与记录管理

建议课内学时 4

目标是在页表、pin、脏位和候选帧之间维持一致性，并解释堆页槽位与记录空间的关系。课堂核心实现 CLOCK；缓冲池调入、脏页回写和堆页作为接口追踪与课后扩展。

题目与接口

源码题目：S01.Q01 页文件读写与分配；S01.Q02 CLOCK 替换；S01.Q04 缓冲池调入与并发替换；S01.Q05 脏页回写与批次跟踪；S01.Q07 堆页槽位与空间组织

先阅读 replacement.cpp、buffer_pool.cpp、frame.hpp 与 heap_page.cpp。列出帧的页号、pin、访问位、脏状态及代次。CLOCK 的输入是候选集合及访问状态；新页映射由缓冲池按同步协议发布。测试表中的操作名表示逻辑事件，实际调用使用对应头文件的类型和返回值。

操作步骤

第一步，画出一个帧从空闲到驻留、可替换、写回和重新分配的状态变化。第二步，在 SO1.Q02 的标记区域实现访问位更新与候选选择，逐项检查 pin 资格。第三步，用 2 帧模型推演下表；若课前已提供独立夹具，将同一序列编码为断言。第四步，再追踪候选结果如何进入 SO1.Q04，分析并发同页调入和失败发布。

检查项	输入或调度	预期事实
BUF01 全 pin	pin A；pin B；请求 C	不替换 A/B；返回无候选或接口定义的失败
BUF02 脏状态合并	修改 A；unpin A；再次 pin/unpin(false)	A 仍需写回，false 不清除已有修改
BUF03 写回失败	A 脏且可淘汰；注入写失败；请求 C	不发布 C 成功映射；A 最新内容未丢失
BUF04 同页调入	两线程同时请求不驻留的 A	只有一个有效驻留映射；各自 pin 正确
HEAP01 槽复用	插入、逻辑删除、满足回收条件后复用	未达到回收条件的版本不被覆盖
HEAP02 空间边界	页剩余空间刚好不足记录与槽开销	插入失败且页结构与可用空间一致

具体淘汰顺序取决于初始时钟指针。若验证具体选择顺序，夹具应显式固定指针、候选次序和访问位；若只验证资格，则对“未选中 pinned 帧”断言。关闭重开测试通过文件 I/O 检查持久化结果。

记录模板

```
step,page,frame,pin,dirty,reference,result
1,A,...,...,...,...,...
# 按事件日志记录实际交错顺序
# 失败时附页表快照和可观察的原始错误
```

常见错误定位

返回旧内容时先检查脏位是否被 false 覆盖，再检查淘汰前写回是否成功。全 pin 时仍返回候选，检查候选算法是否把访问位当作 pin。并发同页出现两帧，检查加载状态的协调和映射发布，而不是只在最终页表外层加锁。

时间与验收

课内 1 学时做状态推演，2 学时实现与调试 CLOCK，1 学时解释失败路径和记录布局。提交区域 diff、至少 BUF01/02 的可重放结果、BUF03/04 的夹具或调度设计，以及页状态表。代码与推演验收使用逐事件状态表。

答辩：为什么重复 unpin 必须被处理？回写期间又发生修改，旧批次完成后能否无条件清 dirty？扩展实现 LRU 并比较访问序列，并检查 storage profile 的组件接线。

L2 B+树与唯一索引

建议课内学时 6

目标是完成键比较和一次叶分裂的实现切片，验证查找、边界扫描与唯一性维护。教师按 S03 的依赖准备页面访问与目录夹具。完整删除合并作为课外扩展。

源码与前置阅读

源码题目：S03.Q01 索引键与节点页算法；S03.Q02 B+Tree 下降与定位；S03.Q03 插入、分裂与新根；S03.Q04 范围扫描与游标；S03.Q06 唯一性与存量冲突检测；S03.Q07 唯一索引等待与重查

从 btree_key、btree_page、btree_insert 与 btree_scan 阅读键、页头、槽项和游标。把“根页发布”“叶链更新”“分隔键传播”分别画出。教师给定 I/O 端口负责页面读写，学生实现分裂和键定位。

手算后实现

使用教学容量 3 推演插入 3、7、9、12：原叶 [3,7,9] 溢出后按课堂示例分为 [3,7] 与 [9,12]，新根分隔键为 9。真实源码测试应按页面实际容量构造足够键，实现按实际页容量计算。记录每次插入后的树高、各页键范围与叶链。

先实现类型化键比较和 lower_bound，再完成下降路径与叶插入；随后处理分裂和新根，最后接入范围终止。修改页前检查容量和所需资源；对失败场景说明哪些内容仍在候选页、哪些已对外发布。

编号	输入	预期结果
IDX01 基本定位	键 3,7,9,12,20；查找 9 和 8	9 可定位，8 不存在
IDX02 半开范围	相同键：[7,20)	7,9,12，恰好 3 个
IDX03 空范围	[8,8)、下界大于上界的接口输入	空结果或明确非法区间错误，遵守接口
IDX04 复合键	(1,20),(2,1),(1,-3)	按逐列有符号比较排序
IDX05 根分裂	插入超过真实页容量的递增和乱序键	所有键可达，树与叶链一致
IDX06 唯一冲突	已有键 7，再插入键 7	失败；原记录与原索引仍一致
IDX07 等待后重查	T1 未结束的键；T2 检查同键	按 T1 结束状态重新判定，不复用过期结论

验证设计

每轮将插入集合排序，与叶链遍历结果比较；对范围结果检查多重集及边界。唯一索引测试同时检查错误结果、堆和索引。等待测试用事件或屏障控制，记录等待前已释放哪些页守卫。比较器遵循键类型和键列顺序。

时间安排与提交

1 学时读页格式与手算，2 学时完成定位和键比较，2 学时实现分裂并核对不变量，1 学时讨论唯一性和验收。提交修改区域、树结构图、IDX01/02/04/05 的运行或待运行断言、一次冲突后的堆索引对照。测试安排见实验状态页。

故障定位与答辩

单键查找正确而范围缺行，优先看叶链和跨页游标；负数顺序错误，检查是否直接比较字节；分裂后旧键丢失，检查父分隔键与槽位置更新。答辩解释 lower_bound 与 upper_bound 的区别，以及待决事务结束后为什么要重查。扩展为删除借位、合并和根收缩时另列测试，不用本实验成绩代替。

L3 Append Unique 与查询执行

建议课内学时 6

本实验实现 Append 与 Unique。S06.Q02 和 S06.Q03 分别提供 6 项、10 项独立测试。完成指定区域后运行具名用例，记录结果并分析失败原因。

准备与定位

源码题目：S06.Q02 Append 流式串接；S06.Q03 全行 Unique

```
mkdir -p course-results/L3
rg -n 'LAB-S06\.Q02:.*-BEGIN' libs
rg -n 'LAB-S06\.Q03:.*-BEGIN' libs
./dev.sh unit S06.Q02 S06.Q03 \
  --report course-results/L3/before.json
```

阅读 docs/course/questions 中两题指导，以及 execution 测试中的 fixed_rows_state.hpp。该夹具提供受控行流。共享的 lowering_select_ops.cpp 只修改 S06.Q03 区域，其他题的空洞保持原样。

任务一实现 Append

先写出 current child、已打开/已访问状态和 EOF 的含义。open 建立本轮状态，不提前打开所有孩子。next 只推进当前孩子；当前 EOF 后关闭或推进到下一孩子，保留行重复次数。发生 open/next/rescan 失败时清理已访问资源，保留原始错误，并清空输出状态。

输入或事件	应观察的行为
A=[1,1,2]，B=[]，C=[2,3]	输出 [1,1,2,2,3]；空孩子不产生行
只请求第一个结果	不提前打开后续孩子
第二个孩子 open 失败	已打开孩子得到清理，第三个不打开
输出部分行后 rescan	按接口重绕已访问孩子，从头重新给出结果

任务二实现 Unique

为完整行定义结构相等与哈希，包含列数、类型、NULL 和值表示。保持第一次出现的行并惰性读取；next 找到一个新行后就返回，不为方便而一次吃完所有输入。已见键需要稳定所有权。rescan 清空集合，再按生命周期重置孩子；失败后按接口限制后续动作。

输入	预期
两列 (NULL::INT,1)，(NULL::INT,1)	只保留第一行
两列 (1,2)，(1,3)	按完整行比较，保留两行
FLOAT -0.0 与 +0.0	按本题位表示规则区分，不套用 SQL =
零列行输入 3 行	Unique 后 1 行；与空输入的 0 行区分
哈希碰撞的两条不同完整行	仍保留两行，哈希值不等于身份

规划侧还要为 SELECT DISTINCT / UNION DISTINCT 包装节点，复制完整输出描述符，并保留显式零列 schema。两个规划侧用例分别检查节点构造和输出描述符。

运行与解释结果

```
./dev.sh unit S06.Q02 S06.Q03 \
  --report course-results/L3/after.json
# 可选：把课程两题 selection 文件复制进 labs/selections 后
./dev.sh selection \
  --file labs/selections/course-append-unique.json
```

查看报告中的实际测试名和数量；Append 应覆盖稳定串接、空输入、重扫及三类失败，Unique 应覆盖 8 个 execution 与 2 个 planner 用例。原始运行日志与 JSON 一起保存。

时间与提交

1 学时推演状态机，2 学时实现 Append，2 学时实现 Unique，1 学时做失败注入与个人答辩。提交 before/after 报告、只含本题区域的 diff、三类比较规则对照、一次资源清理轨迹，以及说明某个测试如何被初始实现触发的短文。

扩展任选表达式求值、TopN、标量聚合或嵌套循环连接，分别参考 S02.Q06、S04.Q05、S05.Q01、S07.Q02。扩展题的测试结果单独记录。答辩现场增加一个空孩子或重复行，先预测结果再运行。

L4 事务锁与死锁处理

建议课内学时 4

目标是实现或验证锁兼容和等待图中的一个切片，并解释死锁解除后系统如何恢复推进。课堂以锁状态表和确定性历史作为代码验收的共同依据。

源码与模型

源码题目：S08.Q04 锁兼容与等待队列；S08.Q05 死锁检测；S08.Q06 提交与中止发布；S08.Q08 事务等待与唤醒

阅读 lock_manager.cpp、lock_manager_state.hpp 与 xid_waiter.cpp。按实际锁模式建立兼容矩阵，列出 granted、waiting、aborted 等状态；涵盖源码定义的完整锁模式。识别等待队列顺序、升级和唤醒条件。

双事务历史

事件	T1	T2	观测
1	取得 A 的排他锁		A 归 T1 持有
2		取得 B 的排他锁	B 归 T2 持有
3	请求 B		T1 等待 T2
4		请求 A	等待图形成环
5	按裁决继续或中止	按裁决继续或中止	一个牺牲者释放资源，幸存者可完成

用测试屏障确保事件 1、2 已完成，再放行 3、4。由屏障记录锁授予事件。牺牲者身份可以由实现策略决定，除非接口明确规定，否则断言应检查环被解除和系统可继续，并记录实际牺牲者。

实施步骤

首先验证兼容锁立即授予、冲突锁进入等待。然后从真实等待关系构造图并检测环，加入三事务长环与无环链。最后跟踪中止后的释放、等待边移除和唤醒；重新执行同一历史，检查没有残留节点。若测试仅验证图算法，将锁管理接线结果单独记录。

编号	场景	预期
LOCK01	两个兼容请求	都获得允许的访问
LOCK02	排他请求与已有冲突锁	按规则等待，不越权授予
LOCK03	T1->T2->T3 无环	不以“有等待”误报死锁
LOCK04	T1->T2->T3->T1	检测环并完成一次有效解除
LOCK05	等待者中止或持有者释放	队列、图与唤醒状态一致
LOCK06	重复运行同一历史	不受上一轮事务 ID 或边影响

提交与验收

1 学时建立状态模型，2 学时实现所选切片与调试，1 学时重放历史和答辩。提交实际调用与事件日志、等待图变化、牺牲者处理轨迹和失败清理说明。提交逐事件状态表，检查环解除、资源释放和幸存事务完成。

故障定位与答辩

检测到环却一直卡住，检查释放和唤醒；没有环也中止，检查是否把兼容关系或过期边加入图；重复运行才失败，检查事务清理。答辩解释等待前释放页守卫的原因，以及无环等待图中的饥饿问题。

L5 快照与多版本可见性

建议课内学时 4

目标是把可见性写成可枚举的判定，区分 SQL 读结果、版本生命周期和唯一索引需要的等待信息。课堂核心为 S09.Q03；RC/SI 配置与快照取得用于解释输入。SQL 调度通过 Session 执行。

源码入口与输入事实

源码题目：S09.Q01 ProcArray 快照与回收边界；S09.Q02 RC/SI 快照协调与配置；S09.Q03 元组可见性与索引版本分类

阅读 visibility.cpp、snapshot_coordinator.cpp、heap_fetch.cpp 与 heap_index_tid.cpp。为每个用例填写当前 XID、命令 ID、创建者、修改者、CLOG 状态、快照上界及取得快照时的活跃集合。先分别分类创建与修改，再组合为可见性；对需要等待的版本保留等待信息。

最小事实表

下表采用教学快照 S：上界为 20，取得时活跃集合为 {12}；读者为事务 18。仅讨论没有删除标记的他人新版本，实际测试按源码类型构造输入。结合事务号、快照活跃集合和提交状态判定可见性。

创建者	当前状态	该快照能否看见	原因
10	快照前提交	可见	已在快照边界内完成
12	现在已提交	不可见	取得 S 时仍活跃
21	现在已提交	不可见	在快照上界之后
11	中止	不可见	创建未生效

为上述每行增加 modifier，分别取当前事务、已提交他人、活跃他人和中止他人，分析旧版本是否仍可见。自己的插入、删除要增加命令 ID 的测试，不仅检查 XID 相等。把版本分类结果传给 fetch/scan 与索引调用方时，保留等待、跟随或跳过所需的信息。

RC 与 SI 的观察

建立一行 x=10。T1 先读到 10；T2 更新为 20 并收到提交成功；T1 再读。在支持的 RC 模式下，新语句快照可读到 20；在 SI 的同一事务快照下，仍读到 10。设置隔离级别的具体 SQL 由项目 parser/session 的实际接口确认，随后将下述事务历史转换为可执行 SQL。

写偏斜历史

```
初始：A=1，B=1，约束 A+B>=1
T1 BEGIN；T2 BEGIN
T1 READ A,B -> 1,1
T2 READ A,B -> 1,1
T1 WRITE A=0
T2 WRITE B=0
T1 COMMIT；T2 COMMIT
观察最终状态与两次提交结果
```

在 SI 下可能得到两个提交成功和 A=B=0，这展示 SI 允许的写偏斜。只有已实现可串行化控制时，才检查系统是否阻止这一非串行历史。另设两事务写同一行的用例，验证写写冲突，二者分开。

提交与答辩

课内 1 学时列事实表，2 学时实现或分析可见性，1 学时重放隔离历史。提交至少 8 组包含创建者与修改者的判定、命令 ID 用例、RC/SI 对照和索引等待分类说明。每组记录完整输入与返回分类，并解释分类依据。

错误只在旧快照出现时检查活跃集合，不只看当前 CLOG。答辩解释“现在已提交”为什么不等于“对旧快照可见”，以及物理 redo 与 SQL 可见性的不同输入和目的。

L6 WAL 重做与恢复验证

建议课内学时 4

目标是理解日志耐久、页 LSN 与事务状态的分工，实现或验证一个 redo 切片，并设计能够证明已确认事务保留的崩溃实验。进程级实验通过完整启动与恢复链执行。

源码入口

源码题目：S10.Q03 WAL 读取与尾部分类；S10.Q08 静态检查点接纳与排空；S10.Q09 堆与 B+Tree 页重做；S10.Q10 事务与目录恢复收尾；S10.Q11 启动分析与恢复调度

先阅读 reader 的尾部分类，再读 heap_startup_redo、btree_startup_redo 与 recovery_page_cache。输入必须是 dispatcher 已验证的日志块；普通 delta、FPI 和 WILL_INIT 采用各自约定的页面读取方式。日志格式沿用项目定义。

单页重做步骤

构造一个可控页面和合法记录，记录应用前 page LSN 与页字节哈希。在候选页上应用更新，验证布局后发布。再次重放同一记录，比较页字节与脏状态；再注入候选页校验失败，确认旧页不变。跨页记录分别检查每页的进度，恢复中断后依靠各页 LSN/FPI 安全续跑。

编号	条件	预期
WAL01	页已覆盖本条 delta	跳过，不重复修改或额外标脏
WAL02	同记录重放两次	最终字节与一次重放相同
WAL03	普通 delta 遇到缺页或坏页	按合同失败，不凭空构造正常页
WAL04	合法 FPI/WILL_INIT	使用规定模式构造并发布候选页
WAL05	事务最后中止但物理记录必要	仍恢复物理布局；SQL 可见性另判
WAL06	日志合法不完整尾与中间损坏	分别处理合法日志尾与中间损坏

专用测试进程的崩溃实验

先创建独立测试目录并记录 PID、启动时间与命令行，确保只操作本实验的数据库进程。客户端逐笔提交带唯一编号的事务，在收到完整成功响应后将编号记入外部确认清单。通过测试屏障或可观察事件选择故障时点，强制终止该专用进程，再用同一测试库重启。

重启后验证确认清单中的每个编号和对应业务值，并检查总行数。还要检查唯一约束、索引与堆一致性、重复重启和 WAL 尾处理。未收到 ACK 的事务可能已经提交，列为结果不确定并进一步核对，按恢复后的提交状态分类。故障注入目标为本实验创建并记录的专用 PID。

```
ack_id,request_id,commit_response_time,expected_value
# 只记录完整收到成功 COMMIT 的事务

run_id,source_commit,fault_event,restart_result
# 原始日志与确认清单保存在测试库目录之外
```

时间安排与提交

1 学时分析 WAL 顺序，2 学时处理页重做或日志尾切片，1 学时检查点与恢复答辩。提交 WAL01/02/05 的输入和断言、故障时点图、候选页失败前后对照。崩溃实验方案包括故障触发、确认清单与重启检查。

故障定位与答辩

ACK 后丢数据，检查日志同步与确认先后；第二次重放改变数据，检查幂等条件和发布 LSN；中止事务恢复后可见，检查事务收尾和 MVCC，而不是删掉它的物理日志。答辩说明组提交如何减少同步次数又不牺牲每个 ACK 的耐久要求。

L7 集成验收与性能分析

建议课内学时 2

本实验把前面的结果整理成可重复的系统评测记录。课堂重点是解释正确性、瓶颈和对照方法。课堂对照可选择已完成的算子切片。

先确认测量层次

```
./dev.sh plan --profile db-basic-rc --json
# 仅在题集、独立测试和能力接线均已补齐后执行
./dev.sh integrate --profile db-basic-rc
```

plan 展开题集与依赖。db-basic-rc、db-relational、db-reference 默认需要 46、76、83 题。单题全部通过后，还要通过真实 Database/Session 的能力接线与 SQL 测试。

一个可解释的对照

选择全排序与 TopN、扫描与索引或连接策略中的一个。固定数据、查询、源码身份、编译器和计时范围，先对比输出多重集，再分别运行至少 3 次并报告中位数及每次原值。记录预热、缓存状态和是否包含初始化。

必须记录	内容
实现身份	源码包哈希、本地提交、工作区是否修改、二进制 SHA256
输入	数据规模与分布、SQL、seed、索引和隔离配置
计时范围	加载、预热、正式执行、结果传输各自是否计入
结果	每次耗时、正确性检查、成功/失败数量、资源观察
结论	变化发生在哪个模块，证据能支持多大范围

决赛扩展入口

grader 在 Linux/Docker 中构建目标 Git 提交并运行 rmdb。学生发放包没有 Git 历史，需要先初始化并提交自己的代码；运行器不会包含未提交修改。Rust 工具链要求以 grader 的 Cargo.toml 为准，当前声明最低 1.88。命令模板如下。

```
cd /path/to/grader
cargo build --release
# 将 REV 替换为目标仓库的完整提交 SHA
REV=your_full_commit_sha
./target/release/grader \
  --repo /absolute/path/to/completed-repo \
  --revision "$REV" --profile quick \
  --seed 1 --work-tmpfs-gib 32
```

quick 仍使用 50 仓库、32 客户端、25 个功能和 5 个恢复用例，以及三轮排名和一致性检查。它只缩短测量时长。相同 seed 下，仓库身份或二进制变化会改变派生工作负载，对照实验应记录每轮派生工作负载身份。Ferrum 使用 Wire 客户端。

提交与综合答辩

0.5 学时检查集成状态，1 学时整理对照与复现步骤，0.5 学时个人答辩。提交一页结果表、所有原始样本、正确性证据和复现命令。算子实验报告耗时、吞吐和资源用量，整库实验报告 NewOrder/min。

答辩围绕一次失败展开：它属于算法、接线、协议、恢复还是测量范围错误？给出能证伪当前解释的下一项检查。比赛要求以相应年度正式发布为准，课堂成绩与比赛排名分别报告。

附录一 实验报告与答辩

实验报告填写顺序

先写完成范围与结论，再给出环境、实现和证据。报告不必复述教材，重点解释自己的决策及其后果。

字段	需要填写的内容
身份	姓名/学号、实验号、题号、源码包哈希、本地提交与未提交差异
合同	输入、输出、状态不变量、允许修改的区域
实现	核心状态、所有权、同步或发布点、失败处理
验证	用例编号、最小输入、预期、实测、命令、日志文件
问题	一个真实失败、定位过程、修复与回归
后续工作	剩余任务、依赖与下一步验证
参考	资料、参考代码与 AI 辅助范围、独立完成部分

结果登记样例

```
实验号: L3
题号: S06.Q02
状态: 通过 / 失败 / 未执行 / 阻塞（选择实际状态）
实际用例数: 由报告读取
命令: 原样粘贴
预期: 输入 A=[1,1], B=[2]，输出 [1,1,2]
实测: 填写运行结果
证据: JSON 报告路径与完整日志
解释: 状态转移与资源清理为何满足合同
```

个人答辩的检查点

能在真实文件中定位自己实现的区域；能在不运行前预测一个新输入；能解释错误码与资源状态；能区分局部测试与整库证据。教师可在已有用例上改变一个边界，如空输入、重复值、失败位置或事务交错，要求现场推演和小幅修改。

材料依据与进一步阅读

课程内容依据下列已收到材料编写。概念算例与实验检查表为课程自编；Ferrum 特定行为以学生版接口、题目指导和测试为准。配套教材课件保留原署名。

编号	材料	使用位置
R1	CST2281 理论大纲与 CST2291 实践大纲	理论 40 学时、实践 32 学时；正式大纲正文优先于文件名
R2	数据库系统概论第6版原课件，第1至13章	关系、SQL、范式、存储、查询、事务和恢复的概念组织
R3	Ferrum-cpp-student labs/manifest.json 与 docs/course/	83 题、实现区域、比较规则、生命周期与独立验收状态
R5	grader README.md、assets/suite_manifest.json、Cargo.toml	运行环境、功能/恢复/TPC-C 范围、协议与提交要求
R6	02_历年赛题下2023至2026年技术方案、决赛题与测试说明	竞赛能力衔接；具体来源与固定版本见来源索引.json

源码接收日期为 2026-10-28。学生版记录的上游 HEAD 为 edd884ccba5b01cb4c3abf054ccdf36da82811d1；完整快照可能含未提交内容，因此以 04_Ferrum实验基础/学生源码清单.json 的文件级 SHA256 为学生发布内容身份。

2027 年赛事规则尚不由本材料规定。历年赛题用于教学训练，正式参赛前应核对当年发布的规则、接口和提交要求。